# Preprocessing

**Filtering, baseline-relative features, and gesture decisions.**

Begin with the GuaqBand preprocessing pipeline.

# 1 · Building GuaqBand

**Build the preprocessing stages of the gesture detector.**

Open `preprocessing/preprocessing_lab.ipynb`.

Implement smoothing, features, distances, and a nearest-neighbor vote.

# What are we building?

Four MLX90393 sensors measure three magnetic-field axes each.

- **12 channels:** s0x, s0y, s0z, through s3z, in µT
- **50 Hz target:** about 20 ms between frames
- **Four labels:** rest, wrist_up, spread, fist

# One frame and one processing history

```
FRAME,sequence,device_us,s0x,s0y,s0z,...,s3z
```

The sequence tracks order. Device time tells us when frames arrived at the sensor stream.

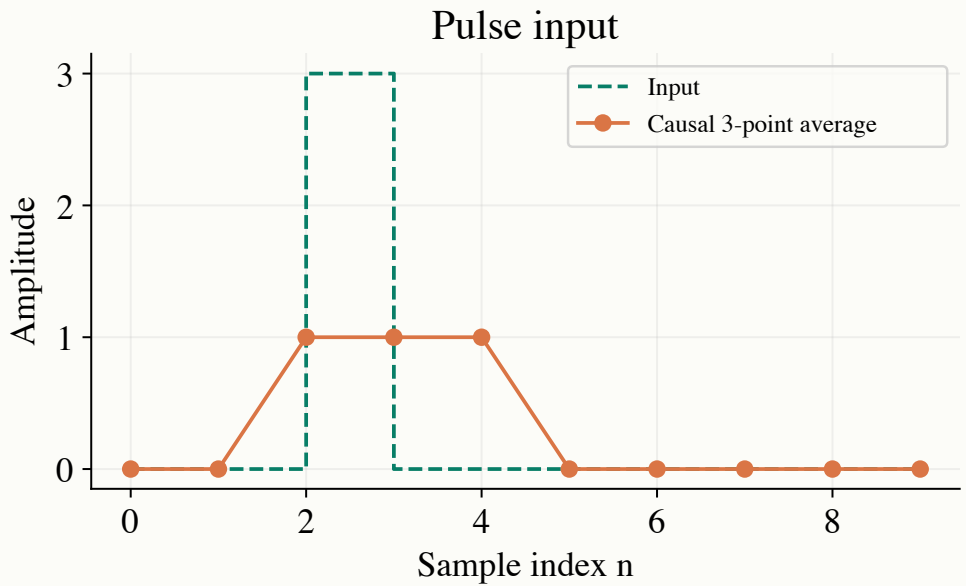
The detector forms a **19 × 12 history** on a uniform 50 Hz grid before computing a feature vector.

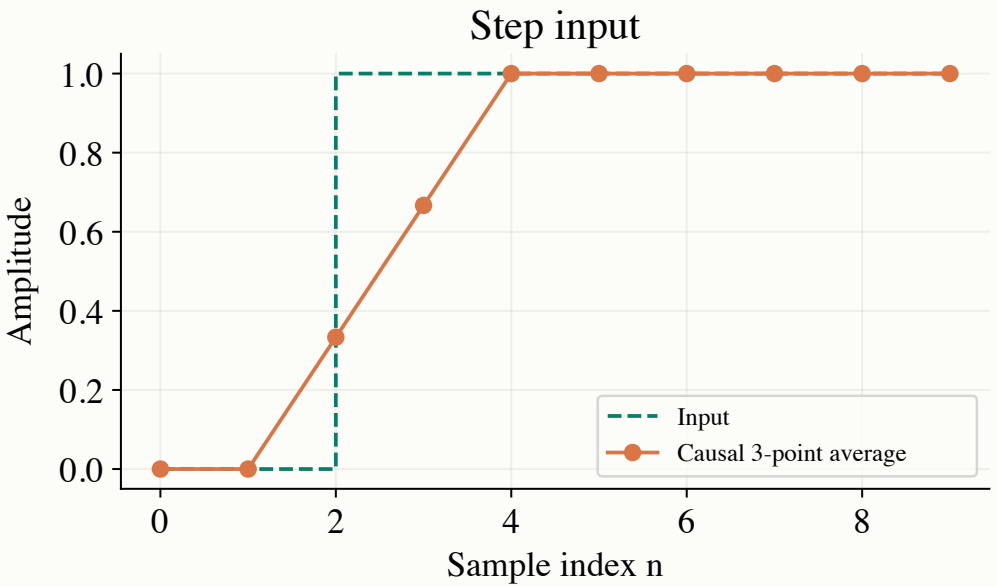
**Reminder**: index ≠ time

# Our pipeline

| Stage | Output |
| --- | --- |
| Timestamp checks and resampling | 19 rows × 12 channels |
| Five-sample causal average | 15 rows × 12 channels |
| Relaxed baseline subtraction | Signed field changes |
| Mean and RMS per channel | 24-value feature vector |
| Training scaling and 3-neighbor vote | Raw gesture label |
| Two matching predictions | Published gesture label |

# Remember the effectiveness of a low pass filter




Sudden changes become wider.

For us, it helps identify when a gesture actually starts.

# Causal averaging

$$y_c[n] = \frac{1}{5} \sum_{k=0}^{4} x_c[n-k]$$

Average over 5 frames.

**What does causal mean?**

# Lab 1 · A causal sensor filter

- Predict the output for a short pulse.
- Implement `moving_average` in notebook **§1**.
- Inspect the 19-input / 15-output plot and causal alignment.

**Return with:** an aligned trace and an explanation of noise reduction versus transition spreading.

# 2 · Features for the armband

**What information should we retain?**

The goal here is effective image compression…

# Relaxed field defines the reference

$$b_c = \text{mean}(\text{known relaxed samples for channel } c)$$

Subtract the baseline after smoothing. Each channel has its own reference.

The live program collects two seconds of relaxed data. Recalibrate when wearer, fit, or sensor placement changes.

A baseline containing a held gesture erases part of that gesture’s displacement.

# Mean preserves direction and RMS preserves size

$$m_c = \mathrm{mean}(y_c - b_c), \qquad r_c = \sqrt{\mathrm{mean}((y_c - b_c)^2)}$$

| Constant field change | Signed mean | RMS |
|---|---|---|
| +3 μT | +3 μT | 3 μT |
| −3 μT | −3 μT | 3 μT |

Save **12 means followed by 12 RMS values** in sensor-axis order.

# Features discard waveform detail

| Filtered field changes in one channel | Mean | RMS |
|---|---|---|
| +1, −1, 0 µT | 0 µT | $\sqrt{2/3}$ µT |
| −1, +1, 0 µT | 0 µT | $\sqrt{2/3}$ µT |

The order changed. The summaries stayed equal.

Signed mean and RMS suit a simple held-pose detector. They cannot preserve every movement’s timing or order.

# Lab 2 · The production feature contract

Implement `feature` in notebook **§2** and compare its outputs with production checks.

- Compare opposite constant changes.
- Contaminate the baseline with a held pose.
- Explain what the window summaries discard.

**Return with:** a feature plot, exact array shapes, and one feature-loss example.

# 3 · Gesture decisions

**How do we turn features into a useful live label?**

Notebook **§3:** implement KNN distances and voting, then inspect the selected neighbors.

# Why scale before finding neighbors?

KNN combines differences across **all 24 features**. Some features naturally vary much more than others.

| Feature | Difference between two examples | Training standard deviation | Scaled difference |
|---|---|---|---|
| A | 20 μT | 20 μT | 1 |
| B | 1 μT | 1 μT | 1 |

Without scaling, A contributes **400** to squared distance; B contributes **1**.

After scaling, both differences are one standard deviation, so each contributes **1**.

**Compare differences relative to each feature's usual variation.**

# A scaled value says how far from usual

$$\text{scaled value} = \frac{\text{value} - \text{training mean}}{\text{training standard deviation}}$$

| Feature | Value | Training mean | Training standard deviation | Scaled value |
|---|---|---|---|---|
| A | 120 | 100 | 20 | 1 |
| B | 3 | 2 | 1 | 1 |

Both values are **one standard deviation above their training average**.

Zero means the training average. Negative values are below it. KNN compares examples on these scaled axes.

# Use the same ruler for every prediction

Compute and save the 24 means and standard deviations from **training examples only**.

Use those same numbers for training, tuning, test, and live inputs.

| Operation | Question it answers |
| --- | --- |
| Relaxed-baseline subtraction | How much did the field change from a relaxed hand? |
| Feature standardization | How large is this feature relative to its training variation? |

# KNN stores labeled examples

Each training row is **24 features + one gesture label**.

A new query is another 24-value vector. Find examples with similar feature values.

**K = 3:** let the three closest examples vote.

# How far is one example from the query?

For standardized training example $z_i$ and standardized query $q$:

$$d_i = \sqrt{\sum_{j=0}^{23} (z_{ij} - q_j)^2}$$

Measure the difference in each of the 24 features, then combine them into one distance.

# See the three nearest neighbors




This illustration uses two features. Your implementation measures distance across **all 24**.

# Three neighbors vote for a gesture

| Neighbor, closest first | Distance | Label |
|---|---|---|
| 1 | 0.3 | fist |
| 2 | 0.4 | rest |
| 3 | 0.6 | fist |

**Prediction: fist**, with two of three votes. The closest example alone does not decide.

If all three labels differ, choose the nearest label.

# Lab 3 · Implement KNN

Notebook **§3:** implement `normalized_distances(model, sample)`.

In the same section, implement `predict(model, sample)`, calling your distance function.

- Use saved training scales and `model['k']`.
- Pass distance, majority-vote, and tie checks.
- Inspect your neighbors, distance bars, and vote plot.

**Return with:** your predicted label and the neighbors supporting it.